ASSIGNMENT 2 – CO2

TASK 1 / Q1: PROCESSES AND PROCESS STATES

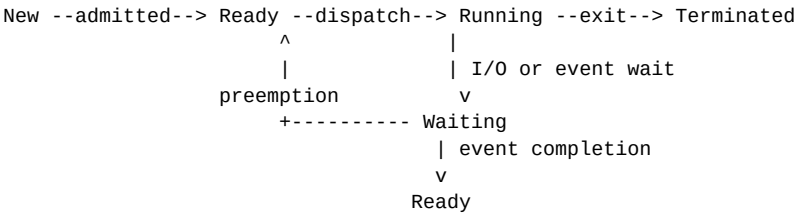
Student Detail	Information
Name	Juhi Surin
Register Number	192525148
Date	03-11-2026
Course (as supplied)	CSA0403 – Computer Vision
Under the Supervision of	Dr. Velmurugan J
Assignment	Assignment 2 – CO2
Task	Q1 – Processes and Process States
Maximum Marks	6
GitHub Repository	CSA04-192525148-Assignment2
GitHub Link	Add your final repository URL here

Course-code verification: The supplied assignment PDF labels Assignment 2 as Operating Systems – CO2. The course detail above is reproduced exactly as provided (CSA0403 – Computer Vision). Please verify the correct course name/code with faculty before final submission.

1. Aim and Requirements

Model process states and the Process Control Block (PCB), implement process creation using fork(), observe processes with ps and /proc, and evaluate whether one process per online-examination student is suitable at a scale of 2,000 concurrent sessions. The assignment expects a state diagram, PCB description, working fork() program with ps evidence and a defended design decision.

2. Process-State Diagram



New: process creation is prepared. **Ready:** eligible for CPU time. **Running:** instructions execute. **Waiting/Blocked:** awaiting an event such as I/O completion. **Terminated:** execution ended; parent collects status using wait()/waitpid().

3. Process Control Block (PCB)

PCB field	Purpose
PID and parent PID	Identify process and parent.
Process state	Record ready, running, waiting or terminated.
Program counter	Identify next instruction.
CPU registers	Preserve execution context during context switches.
Scheduling information	Priority, scheduling parameters and queue links.
Memory information	Address-space and page-table references.
I/O and file information	Open files, devices and pending I/O.
Accounting/security	CPU usage, user identity and permissions.

The PCB is maintained by the operating system. It allows the kernel to suspend a process, preserve its execution context and later resume it.

4. Implementation: fork(), ps and /proc

The accompanying process_states.c creates three child processes. Each child sleeps for 12 seconds so it can be observed. The parent reads Name, State, PID, PPID and Threads from /proc/<PID>/status, then waits for each child using waitpid().

```
gcc -std=c11 -Wall -Wextra -Wpedantic process_states.c -o process_states
./process_states
```

While children sleep, use the actual PIDs printed by your run:

```
ps -o pid,ppid,stat,comm -p <PID1>,<PID2>,<PID3>
cat /proc/<PID1>/status
```

A State value of S indicates interruptible sleeping. Capture your own screenshots during the 12-second interval; PIDs change on each run.

5. Execution Evidence and Environment

The included sample_execution_output.txt records the test run. It shows the child PIDs, parent PIDs, sleeping state and successful waitpid() reaping. The test environment was GCC 14.2.0, Debian GNU/Linux 13 (trixie), Linux kernel 6.18.57. Verify and report your own environment if it differs.

Evidence limitation: Add screenshots captured on your own machine. In particular, run ps while children are alive; the saved text output alone is not a substitute for requested screenshots.

6. Evaluation for 2,000 Concurrent Sessions

One process per student offers strong address-space isolation, but 2,000 processes create many process records, separate virtual address spaces and scheduling operations. Context switches can reduce useful CPU time. I/O-blocked sessions may not use CPU but still occupy process resources.

A thread per student usually has lower memory and creation overhead because threads share code, data and open resources; each still has its own stack and execution context. Shared state requires synchronization, and a process-wide failure can affect all its threads.

A fixed worker-process pool bounds the number of active worker processes and queues incoming sessions. It balances worker overhead and process-level isolation, but queue latency, pool size and load balancing must be evaluated.

Design	Advantages	Limitations
One process/student	Strong address-space isolation.	High memory/kernel bookkeeping and context-switch overhead.
One thread/student	Lower per-session overhead; shared resources.	Synchronization required; shared process failure risk.
Fixed worker pool	Bounded worker count; worker-level isolation.	Queueing delay; pool sizing and utilization matter.

Recommendation: Use a bounded worker-process pool as the baseline, potentially with asynchronous I/O or a modest number of threads per worker. Select the pool size using load tests measuring response latency, throughput, memory, CPU utilization and failure recovery.

7. Alternative Concept and Complexity Analysis

For n student sessions, one-process-per-student has O(n) process records and separate virtual address spaces. One-thread-per-student has O(n) thread records/stacks but shares an address space. A fixed pool of k workers has O(k) worker processes plus O(n) session/queue metadata. Creating n processes or threads requires O(n) creation operations; creating k workers is O(k). A suitable queue may provide O(1) enqueue/dequeue. These are asymptotic estimates, not measured timings.

Process context switches can involve address-space and TLB effects, whereas same-process thread switches may avoid changing address spaces. Actual costs depend on hardware, kernel, workload and scheduler; do not claim fixed timing without measurements.

8. Conclusion

The implementation demonstrates child process creation with fork(), observation of sleeping processes through /proc and collection of child exit statuses using waitpid(). For 2,000 examination sessions, a bounded worker-process pool is a defensible choice because it limits worker-process overhead while retaining process-level isolation. A hybrid worker-plus-thread or asynchronous-I/O design can improve concurrency when synchronization and load testing are handled correctly.

9. Final Submission Checklist

1. Add your compilation/execution screenshot.
2. Add a ps screenshot while children are alive.
3. Add a /proc/<PID>/status screenshot.
4. Replace the GitHub URL placeholder with the actual repository URL.
5. Confirm the course name/code with faculty.
6. Be ready to explain the code and design trade-offs during viva.

10. References

- Assignment 2 – CO2, Q1: Processes and Process States, supplied assignment PDF, page 12.
- Linux manual pages: fork(2), waitpid(2), ps(1), proc(5).